LeetCode Practice

19 Easy: Arrays & Hashing

Question, examples, then the six-phase interview approach with the brute force in phase 2

A split copy of the full All Questions book; every feature and link works the same

19
EASY

Patterns (in order)

1. Arrays & Hashing (19)

How to use

- Tap any title in the Contents to jump to it. A topic name opens that topic's own divider page.
- Every page has < Prev | Contents | Next > at the top.
- Prev and Next move one question at a time; Contents returns here. Each topic starts on its own page.
- Select code and press Cmd/Ctrl+C to copy it.

Go to Contents >

Contents

19 questions | tap a title to jump to it

Easy questions (19) Go >

Arrays & Hashing (19) Go >

E	1. Two Sum	LC 1
E	2. Missing Ranges	LC 163
E	3. Summary Ranges	LC 228
E	4. Range Sum Query - Immutable	LC 303
E	5. Moving Average from Data Stream	LC 346
E	6. Logger Rate Limiter	LC 359
E	7. Find All Numbers Disappeared in an Array	LC 448
E	8. Max Consecutive Ones	LC 485
E	9. Design HashSet	LC 705
E	10. Find Pivot Index	LC 724
E	11. Find Anagram Mappings	LC 760
E	12. Verifying an Alien Dictionary	LC 953
E	13. Find the Town Judge	LC 997
E	14. Counting Elements	LC 1426
E	15. Shuffle the Array	LC 1470
E	16. Running Sum of 1d Array	LC 1480
E	17. Count Good Triplets	LC 1534
E	18. Maximum Population Year	LC 1854
E	19. Concatenation of Array	LC 1929

Easy questions (19)

EASY | TOPIC 1 OF 1

Arrays & Hashing

19 Easy questions start on the next page

Before this: this is the first topic
After this: this is the last topic

IN THIS TOPIC

1. Two Sum LC 1
2. Missing Ranges LC 163
3. Summary Ranges LC 228
4. Range Sum Query - Immutable LC 303
5. Moving Average from Data Stream LC 346
6. Logger Rate Limiter LC 359
7. Find All Numbers Disappeared in an Array LC 448
8. Max Consecutive Ones LC 485
9. Design HashSet LC 705
10. Find Pivot Index LC 724
11. Find Anagram Mappings LC 760
12. Verifying an Alien Dictionary LC 953
13. Find the Town Judge LC 997
14. Counting Elements LC 1426
15. Shuffle the Array LC 1470
16. Running Sum of 1d Array LC 1480
17. Count Good Triplets LC 1534
18. Maximum Population Year LC 1854
19. Concatenation of Array LC 1929

EASY | Arrays & Hashing | LeetCode #1 | 1 of 19 | 1/19

Two Sum

QUESTION

Given an array of integers nums and an integer target, return indices of the two numbers such that they add up to target.

You may assume that each input would have exactly one solution, and you may not use the same element twice.

You can return the answer in any order.

Example 1	
Input	`nums = [2,7,11,15], target = 9`
Output	`[0,1]`
Explanation	*Because nums[0] + nums[1] == 9, we return [0, 1].*

Example 2	
Input	`nums = [3,2,4], target = 6`
Output	`[1,2]`

Example 3	
Input	`nums = [3,3], target = 6`
Output	`[0,1]`

CONSTRAINTS

- 2 <= nums.length <= 10^4
- 10^9 <= nums[i] <= 10^9
- 10^9 <= target <= 10^9
- Only one valid answer exists.
- Follow-up: Can you come up with an algorithm that is less than $O(n^2)$ time complexity?

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We're given an array of integers and a target value,

and we need to return the indices of the two numbers

that add up to the target — not the values, the indices. Right?"

"A few quick questions:

Can the same element be used twice? I'm guessing no.

Is there always exactly one valid answer?

Can the array be empty or have only one element?

Are there negative numbers or duplicates?"

"Let me walk the example: nums = [2, 7, 11, 15], target = 9.

2 + 7 = 9, so we return [0, 1]. Got it.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        if nums[i] + nums[j] == target:
            return [i, j]
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Checks all n(n-1)/2 pairs. A hash map of the values seen so far finds each number's partner in O(1), so one pass is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is searching the rest of the array for a complement each time.

I notice we're repeatedly asking: does the number I need exist?

A hash map answers that in O(1).

Walk forward through the array. For each number, compute complement = target - num.

If complement is already in the map, return both indices.

Otherwise, store the current number and index and keep going.

Pseudocode:

```
seen = {}
for index, num in nums:
complement = target - num
if complement in seen: return [seen[complement],
  index]
seen[num] = index
```

Time: O(n). Space: O(n). Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1_TwoSum:
    def twoSum(self, nums: List[int], target: int) -> List
[int]:
        seen = {}                                # maps value
-> index
        for index, num in enumerate(nums):
            complement = target - num
            if complement in seen:
                return [seen[complement], index]
            seen[num] = index
        return []
```

PHASE 5 TEST & DEBUG

"Let me trace twoSum on [2,7,11,15], target=9."

seen={}, i=0: num=2, need=9-2=7, 7 not in seen -> seen={2:0}

i=1: num=7, need=9-7=2, 2 in seen at index 0 -> return [0,1] ok

WHY hash map: O(n) lookup instead of nested loop O(n²).

Edge: [3,3], target=6 -> i=0: seen={3:0}; i=1: need=3, found at 0 != 1 -> [0,1] ok

Call trace: one-pass, exit on first match.

Contrast: brute force tries all pairs (i,j); hash map exits immediately.

"Result [0,1] because nums[0]+nums[1]=2+7=9 ok"

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
one pass	worst case every element enters the map

Trade-off: O(1) space would require O(n²) time — no free lunch here.

If the array were sorted I'd use two pointers: O(n) time, O(1) space.

But we need original indices and the input is unsorted, so hash map wins.

Given more time I'd ask: what if there are multiple valid pairs?

We'd collect all of them rather than returning on the first hit.

EASY | Arrays & Hashing | LeetCode #163 | 2 of 19 | 2/19

Missing Ranges

QUESTION

You are given an inclusive range [lower, upper] and a sorted unique integer array nums, where all elements are within the inclusive range.

A number x is considered missing if x is in the range [lower, upper] and x is not in nums.

Return the shortest sorted list of ranges that exactly covers all the missing numbers. That is, no element of nums is included in any of the ranges, and each missing number is covered by one of the ranges.

Example 1

Input	`nums = [0,1,3,50,75], lower = 0, upper = 99`
Output	`[[2,2],[4,49],[51,74],[76,99]]`
Explanation	*The ranges are:* *[2,2]* *[4,49]* *[51,74]* *[76,99]*

Example 2

Input	`nums = [-1], lower = -1, upper = -1`
Output	`[]`
Explanation	*There are no missing ranges since there are no missing numbers.*

CONSTRAINTS

- 10^9 <= lower <= upper <= 10^9
- 0 <= nums.length <= 100
- lower <= nums[i] <= upper
- All the values of nums are unique.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We have a sorted array of unique integers, and an inclusive range [lower, upper].

We need to return the list of ranges that cover every number

inside [lower, upper] that does NOT appear in nums. Right?"

"A few quick questions:

Can nums be empty? If so I assume the whole [lower, upper] is one missing range.

Are all elements in nums guaranteed to be within [lower, upper]? Good.

The output ranges should be as compact as possible — so [2,2] not just [2]?"

PHASE 1 CLARIFY

"Let me walk the example:

nums=[0,1,3,50,75], lower=0, upper=99.

0 and 1 are present so no gap at the start.

3 is present but 2 is missing -> [2,2].

50 is present but 4–49 are missing -> [4,49].

75 is present but 51–74 are missing -> [51,74].

76–99 are all missing -> [76,99].

Output: [[2,2],[4,49],[51,74],[76,99]]. Got it.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for v in vs:
    # from lower to upper
    if v is not in nums:
        add it to the current missing range,
            else close the range
```

TIME	SPACE
O(upper - lower + n)	O(n)

WHY IT IS SLOW

Walks every value in the range, which can be about 2^31. Only the gaps between neighbouring numbers (and the two ends) matter: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The brute force iterates over the entire numerical range — up to 2 billion steps.

But nums has at most 100 elements. The gaps I care about

only exist between consecutive numbers in nums, before the first, and after the last.

So I only need to check those boundaries — that's O(n), not O(range).

I think of nums as a set of walls. The missing ranges are the gaps between walls.

There are at most n+1 gaps: one before nums[0], one between each pair, one after nums[-1].

Pseudocode:

```
if nums is empty: return [[lower, upper]]
if lower < nums[0]: add [lower, nums[0]-1]
for each consecutive pair (prev, curr) in nums:
if prev+1 < curr: add [prev+1, curr-1]
if nums[-1] < upper: add [nums[-1]+1, upper]
```

Time: O(n). Space: O(1) extra. Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach.

I'll use pairwise from itertools to cleanly iterate consecutive pairs.

```
class _163_MissingRanges:
    def findMissingRanges(self, nums: List[int], lower: in
t, upper: int) -> List[List[int]]:
        if not nums:
            return [[lower, upper]]

        result = []
        first, last = nums[0], nums[-1]

        if lower < first:
            result.append([lower, first - 1])

        for prev, curr in pairwise(nums):
            if prev + 1 < curr:
                result.append([prev + 1, curr - 1])

        if last < upper:
            result.append([last + 1, upper])

        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

nums=[0,1,3,50,75], lower=0, upper=99

nums not empty. first=0, last=75.

lower(0) < first(0)? No. No leading gap.

pairwise: (0,1) -> 1 < 1? No. (1,3) -> 2 < 3? Yes -> [2,2]. (3,50) -> 4 < 50? Yes -> [4,49]. (50,75) -> 51 < 75? Yes -> [51,74].

last(75) < upper(99)? Yes -> [76,99].

result = [[2,2],[4,49],[51,74],[76,99]] ok

nums=[-1], lower=-1, upper=-1

first=-1, last=-1.

lower(-1) < first(-1)? No.

pairwise: empty (only one element).

last(-1) < upper(-1)? No.

result = [] ok

nums=[], lower=1, upper=5

Empty -> return [[1,5]] ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)**
one pass through nums plus constant work per gap	extra — result list aside, no auxiliary storage

This is a massive improvement over O(upper - lower) brute force given the constraint that nums has at most 100 elements.

Trade-off: if nums were not sorted I'd need to sort first — O(n log n).

The problem guarantees sorted input so we skip that.

PHASE 6 COMPLEXITY & FOLLOW-UP

pairwise is Python 3.10+. If the interviewer is on an older version

I'd write: for i in range(1, len(nums)): prev, curr = nums[i-1], nums[i].

Given more time I'd ask: what if the output should be strings like '2->2'

instead of lists? Just a formatting change at the append step.

IN YOUR APP | Expose | Editorial | Community

EASY | Arrays & Hashing | LeetCode #228 | 3 of 19 | 3/19

Summary Ranges

QUESTION

You are given a sorted unique integer array nums.

A range [a,b] is the set of all integers from a to b (inclusive).

Return the smallest sorted list of ranges that cover all the numbers in the array exactly. That is, each element of nums is covered by exactly one of the ranges, and there is no integer x such that x is in one of the ranges but not in nums.

Each range [a,b] in the list should be output as:

CONSTRAINTS

- "a->b" if a != b
- "a" if a == b

Example 1

Input	`nums = [0,1,2,4,5,7]`
Output	`["0->2","4->5","7"]`
Explanation	*The ranges are:* *[0,2] --> "0->2"* *[4,5] --> "4->5"* *[7,7] --> "7"*

Example 2

Input	`nums = [0,2,3,4,6,8,9]`
Output	`["0","2->4","6","8->9"]`
Explanation	*The ranges are:* *[0,0] --> "0"* *[2,4] --> "2->4"* *[6,6] --> "6"* *[8,9] --> "8->9"*

- 0 <= nums.length <= 20
- 2^{31} <= nums[i] <= 2^{31} - 1
- All the values of nums are unique.
- nums is sorted in ascending order.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given sorted unique integers, return the smallest sorted list of ranges that cover exactly all numbers.

Consecutive numbers in same range; non-consecutive start a new range. Right?"

"A few quick questions:

Single element ranges: '1'? Multi: '1->4'? Yes."

"Let me walk: nums=[0,1,2,4,5,7] -> ['0->2','4->5','7'] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for i in is:
        if nums[j] - nums[i] == j - i keep j
    if i starts a range:
        output nums[i] to nums[the last kept j]
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Re-extends ranges from every position. One pass that closes a range when the next number is not previous + 1 is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Single scan grouping consecutive runs — O(n). Walk-through:"

`i = 0. While i < n:`

`start = nums[i].`

While i+1 < n and nums[i+1] == nums[i]+1: i++ (extend the run).

`end = nums[i].`

If start == end: append str(start).

Else: append str(start) + '->' + str(end).

i++.

HOW runs are detected: advance i while consecutive elements differ by exactly 1.

When the run breaks (gap > 1): record the range [start, end].

WHY single scan suffices: nums is already sorted -> consecutive elements form ranges.

Time: O(n). Space: O(1) extra.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _228_SummaryRanges:
    def summaryRanges(self, nums):
        result = []
        i = 0
        while i < len(nums):
            start = nums[i]
            while i + 1 < len(nums) and nums[i + 1] == num
s[i] + 1:
                i += 1
            end = nums[i]
            result.append(str(start) if start == end else
f'{start}->{end}')
            i += 1
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

nums=[0,1,2,4,5,7]:

i=0(0): extend to 1,2. end=2. append '0->2'. i=3(4): extend to 5. end=5. append '4->5'. i=5(7): end=7. append '7'. -> ['0->2','4->5','7'] ok

nums=[]: return [] ok. nums=[1,3]: ['1','3'] ok

Inner while correctly extends a consecutive run

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(1)** extra

Follow-up: Missing Ranges (#163) — return ranges of MISSING numbers given [lower,upper].

Follow-up: Merge Intervals (#56) — similar range merging but with arbitrary overlap.

EASY | Arrays & Hashing | LeetCode #303 | 4 of 19 | 4/19

Range Sum Query - Immutable

QUESTION

Given an integer array nums, handle multiple queries of the following type:

CONSTRAINTS

- 1. Calculate the sum of the elements of nums between indices left and right inclusive where left <= right.

Implement the NumArray class:

- NumArray(int[] nums) Initializes the object with the integer array nums.
- int sumRange(int left, int right) Returns the sum of the elements of nums between indices left and right inclusive (i.e. nums[left] + nums[left + 1] + ... + nums[right]).
- 1 <= nums.length <= 10^4
- 10^5 <= nums[i] <= 10^5
- 0 <= left <= right < nums.length
- At most 10^4 calls will be made to sumRange.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Immutable array; answer range sum queries [left, right] repeatedly.

Precompute prefix sums for O(1) per query. Right?"

"A few quick questions:

Multiple queries? Yes — precompute once, answer in O(1) each time."

"Let me walk: nums=[-2,0,3,-5,2,-1]. sumRange(0,2)=-2+0+3=1 ok. sumRange(2,5)=-1 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
def sumRange(l, r):
    add nums[l] through nums[r] with a loop
```

TIME	SPACE
O(n) per query	O(1)

WHY IT IS SLOW

Every query rescans its range. Prefix sums (prefix[i + 1] = prefix[i] + nums[i]) answer each query as prefix[r + 1] - prefix[l] in O(1) after an O(n) build.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Prefix sum array built once, O(1) query — O(1) query after O(n) build. Walk-through:"

Build: prefix[0]=0. prefix[i] = prefix[i-1] + nums[i-1]. (1-indexed prefix sums)

Query sumRange(l, r): return prefix[r+1] - prefix[l].

WHY prefix[r+1]-prefix[l]: prefix[r+1] = sum of nums[0..r]. prefix[l] = sum of nums[0..l-1].

Subtracting = sum of nums[l..r].

WHY 0-pad at prefix[0]: avoids special case when l==0 (prefix[0]=0 -> correct subtraction).

Build once, answer many queries in O(1) each.

Time: O(n) build, O(1) query. Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _303_RangeSumQueryImmutable:
    def __init__(self, nums):
        self._prefix = [0] * (len(nums) + 1)
        for i, v in enumerate(nums):
            self._prefix[i + 1] = self._prefix[i] + v

    def sumRange(self, left, right):
        return self._prefix[right + 1] - self._prefix[left
]
```

PHASE 5 TEST & DEBUG

"Let me trace NumArray([-2,0,3,-5,2,-1]) then sumRange(0,2) and sumRange(2,5)."

Build prefix: prefix=[0,-2,-2,1,-4,-2,-3]

sumRange(0,2)=prefix[3]-prefix[0]=1-0=1 ok (-2+0+3=1)

sumRange(2,5)=prefix[6]-prefix[2]=-3-(-2)=-1 ok (3-5+2-1=-1)

WHY prefix sum: O(1) query after O(n) build; avoids re-summing each query.

prefix[i] = sum of nums[0..i-1]; range sum = prefix[r+1]-prefix[l].

Edge: sumRange(0,0) = prefix[1]-prefix[0] = nums[0] ok

Build O(n), query O(1); useful when many queries on same static array.

PHASE 6 COMPLEXITY & FOLLOW-UP

Init O(n). Query O(1). Space O(n).

Follow-up: Range Sum Query — Mutable (#307) — needs Fenwick tree or segment tree.

Follow-up: 2D prefix sums — extend to matrix[r1..r2][c1..c2] queries.

EASY | Arrays & Hashing | LeetCode #346 | 5 of 19 | 5/19

Moving Average from Data Stream

QUESTION

Given a stream of integers and a window size, calculate the moving average of all integers in the sliding window.

Implement the MovingAverage class:

CONSTRAINTS

- MovingAverage(int size) Initializes the object with the size of the window size.
- double next(int val) Returns the moving average of the last size values of the stream.
- 1 <= size <= 1000
- 10^5 <= val <= 10^5
- At most 10^4 calls will be made to next.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We're designing a class that receives a stream of integers one at a time.

Each call to next() should return the average of the last 'size' values seen so far.

If fewer than 'size' values have arrived, we average however many we have. Right?"

"A few quick questions:

Can size be 1? Constraints say yes — that's just returning the latest value.

Can values be negative? Yes, constraints say down to -10^5.

Is there a limit on how many next() calls we get? Up to 10^4 — good to know for space."

"Let me walk the example:

size=3. next(1)=1.0 (only 1 value). next(10)=5.5 (avg of 1,10).

```
next(3)=4.667 (avg of 1,10,3 — window full now).
next(5)=6.0 (avg of 10,3,5 — 1 falls out). Got it.
```

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
result.append(every_value)
def next(val):
    average the last size values by summing them
        from scratch
```

TIME	SPACE
O(size) per call	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Re-sums the window on every call and keeps all history. Keep a queue of the last size values and a running sum (add the new, subtract the one that leaves): O(1) per call, O(size) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Two inefficiencies in the brute force:

First, we recompute the sum from scratch every call — that's O(size) work.

Second, we store the entire history but only ever need the last 'size' values.

I notice we're recomputing the same sum repeatedly. What if we cache it?

I'll keep a running sum and update it incrementally:

when a new value comes in, add it. When the window is full,

subtract the oldest value before adding the new one.

For the window itself, a deque is perfect — O(1) append on the right,

O(1) popleft when the oldest value falls out. The deque never exceeds 'size' elements.

Pseudocode:

__init__: q = deque(), running_sum = 0, self.size = size

```
next(val):
if len(q) == size: running_sum -= q.popleft()
running_sum += val
q.append(val)
return running_sum / len(q)
```

Time: O(1) per next() call. Space: O(size) — the deque is capped.

Does that make sense before I write it out?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _346_MovingAverage:
    def __init__(self, size: int):
        self.size = size
        self.window = deque()
        self.running_sum = 0

    def next(self, val: int) -> float:
        if len(self.window) == self.size:
            self.running_sum -= self.window.popleft()   # evict oldest
        self.running_sum += val
        self.window.append(val)
        return self.running_sum / len(self.window)
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

"Let me trace through the example: size=3."

`next(1): window not full. sum=1, window=[1]. return 1/1 = 1.0 ok`

`next(10): window not full. sum=11, window=[1,10]. return 11/2 = 5.5 ok`

`next(3): window not full. sum=14, window=[1,10,3]. return 14/3 = 4.667 ok`

`next(5): window full -> popleft() removes 1, sum=14-1=13.`

sum=13+5=18, window=[10,3,5]. return 18/3 = 6.0 ok

Edge — size=1:

`next(7): window full immediately after first call? No — starts empty.`

sum=7, window=[7]. return 7/1 = 7.0 ok

`next(4): window full (size=1). popleft removes 7, sum=0+4=4, window=[4].`

`return 4/1 = 4.0 ok`

Running sum stays consistent across evictions

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(1)	**O(size)**
per next() call — all operations are constant: deque append,	the deque holds at most 'size' elements regardless of

popleft, addition, division.

how many total calls are made. Much better than the O(n) brute force.

Trade-off worth mentioning: floating point precision. If values are large

and size is large, the running sum can accumulate rounding errors over time.

A production system might use Python's Decimal or periodically recompute

the sum from scratch to correct drift.

Alternative: a circular buffer (fixed-size list with a pointer) does the same

thing with slightly better cache locality than a deque, but the deque is

cleaner to read and interview-safe.

Given more time I'd ask: what if we need to support a variable window size

that can change between calls? We'd need to resize the deque dynamically.

EASY | Arrays & Hashing | LeetCode #359 | 6 of 19 | 6/19

Logger Rate Limiter

QUESTION

Design a logger system that receives a stream of messages along with their timestamps. Each unique message should only be printed at most every 10 seconds (i.e. a message printed at timestamp t will prevent other identical messages from being printed until timestamp t + 10).

All messages will come in chronological order. Several messages may arrive at the same timestamp.

Implement the Logger class:

CONSTRAINTS

- Logger() Initializes the logger object.
- bool shouldPrintMessage(int timestamp, string message) Returns true if the message should be printed in the given timestamp, otherwise returns false.
- 0 <= timestamp <= 10^9
- Every timestamp will be passed in non-decreasing order (chronological order).
- 1 <= message.length <= 30
- At most 10^4 calls will be made to shouldPrintMessage.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

`shouldPrintMessage(timestamp, message): return true if message hasn't been printed in last 10 seconds. Right?"`

"A few quick questions:

Timestamps arrive in non-decreasing order? Yes. Same message can be printed again after 10 seconds? Yes."

"Let me walk: (1,'foo')->T,(2,'bar')->T,(3,'foo')->F,(11,'foo')->T ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
shown = a list of (time, message) already displayed
for message in messages:
    look backward in shown for the last time this
        message was printed
print only if it was at least 10 seconds ago
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

Scans the history for every call. A hash map from message to the next time it may be printed makes each call O(1).

PHASE 2 BRUTE FORCE

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Hash map + queue with expiry — O(1) per call. Walk-through:"

Store (message, timestamp) pairs. When shouldPrintMessage(t, msg) is called:

If msg not in timestamps OR t - timestamps[msg] >= 10:

`timestamps[msg] = t; return True.`

Else: return False.

WHY O(1): hash map lookup and update are O(1).

WHY >= 10: message can be printed again 10 seconds or more after the last print.

No cleanup needed: old timestamps are simply overwritten when a message is allowed again.

Time: O(1). Space: O(unique messages).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _359_LoggerRateLimiter:
    def __init__(self):
        self._last_printed = {}

    def shouldPrintMessage(self, timestamp, message):
        last = self._last_printed.get(message, -10)
        if timestamp - last >= 10:
            self._last_printed[message] = timestamp
            return True
        return False
```

PHASE 5 TEST & DEBUG

"Let me trace Logger shouldPrintMessage(1,'foo'), (2,'bar'), (3,'foo'), (11,'foo')."

Store msg->timestamp_last_printed

t=1, 'foo': not in log -> print, log={'foo':1} -> True

t=2, 'bar': not in log -> print, log={'foo':1,'bar':2} -> True

t=3, 'foo': log['foo']=1, 3-1=2 < 10 -> False

t=11, 'foo': log['foo']=1, 11-1=10 >= 10 -> print, log={'foo':11,'bar':2} -> True ok

WHY hash map: O(1) lookup for last print time of each message.

Edge: first call always prints; rate-limit window is exactly 10 seconds.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(1)	O(distinct messages in last 10s)

Follow-up: Rate Limiter with expiry (LRU-style) — evict on access using OrderedDict.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: Hit Counter (#362) — count hits in last 300s; use deque.

IN YOUR APP | Expose | Editorial | Community

EASY | Arrays & Hashing | LeetCode #448 | 7 of 19 | 7/19

Find All Numbers Disappeared in an Array

QUESTION

Given an array nums of n integers where nums[i] is in the range [1, n], return an array of all the integers in the range [1, n] that do not appear in nums.

Example 1	
Input	`nums = [4,3,2,7,8,2,3,1]`
Output	`[5,6]`

Example 2	
Input	`nums = [1,1]`
Output	`[2]`

CONSTRAINTS

- n == nums.length
- 1 <= n <= 10^5
- 1 <= nums[i] <= n
- Follow up: Could you do it without extra space and in O(n) runtime? You may assume the returned list does not count as extra space.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given an array of n integers in range [1,n], find all integers from 1 to n that don't appear.

Return the missing numbers. O(n) time, O(1) extra space (output list doesn't count). Right?"

"A few quick questions:

Can duplicates appear? Yes — some numbers appear twice, others zero times."

"Let me walk: nums=[4,3,2,7,8,2,3,1] -> missing: 5,6 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for v in range(1,_n_+_1):
    scan the array for v
    collect the values that are missing
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

A scan per value. Mark presence by negating nums[abs(v) - 1], then every index that is still positive is a missing number: O(n) time and O(1) space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the O(n) extra set.

Use the array itself as a hash map: for each value v = abs(nums[i]),

negate nums[v-1] to mark v as 'seen'. Then collect indices where value is still positive.

Time: O(n). Space: O(1) extra."

Key insight: the hash map provides O(1) lookup — a single pass over the data replaces the O(n) inner scan, reducing total complexity by an order of magnitude.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _448_FindAllNumbersDisappeared:
    def findDisappearedNumbers(self, nums):
        for num in nums:
            idx = abs(num) - 1
            nums[idx] = -abs(nums[idx])
        return [i + 1 for i, v in enumerate(nums) if v > 0
]
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

nums=[4,3,2,7,8,2,3,1]:

num=4: nums[3]=-7. num=3: nums[2]=-2. num=2: nums[1]=-3. num=7: nums[6]=-3.

num=8: nums[7]=-1. num=2: idx=1, nums[1]=-3 (already negative, negate abs -> -3).

num=3: nums[2]=-2 (mark). num=1: nums[0]=-4.

Positives at indices 4,5 (values 8,-2) -> wait, nums[4]=8 positive -> 5, nums[5]=2 positive -> 6.

-> [5,6] ok

Abs() handles already-negated values; final scan finds unmarked positions

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(1) extra — modifies input array

This is a classic 'use array as implicit hash map' trick for [1,n] valued arrays.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: Find the Duplicate Number (#287) — one duplicate; Floyd's cycle detection.

Follow-up: First Missing Positive (#41) — arbitrary values, uses cyclic sort.

IN YOUR APP | Expose | Editorial | Community

EASY | Arrays & Hashing | LeetCode #485 | 8 of 19 | 8/19

Max Consecutive Ones

QUESTION

Given a binary array nums, return the maximum number of consecutive 1's in the array.

Example 1	
Input	`nums = [1,1,0,1,1,1]`
Output	`3`
Explanation	*The first two digits or the last three digits are consecutive 1s. The maximum number of consecutive 1s is 3.*

Example 2	
Input	`nums = [1,0,1,1,0,1]`
Output	`2`

CONSTRAINTS

- 1 <= nums.length <= 10^5
- nums[i] is either 0 or 1.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a binary array of only 0s and 1s, return the length of the longest

run of consecutive 1s. If there are no 1s we return 0. Right?"

"A few quick questions:

Can the array be empty?

Is it guaranteed to contain only 0 and 1?"

"Let me trace: nums=[1,1,0,1,1,1].

Runs: [1,1] length 2, then [1,1,1] length 3. Answer = 3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    count the run of 1s that starts there
    best = max(best, candidate)
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

A run of length n is re-counted from every position inside it. One pass with a running counter (reset on 0) is O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the nested restart. Instead I keep a running streak:

increment when I see a 1, reset to 0 on a 0, and update the global best every step.

One pass, no inner loop.

Pseudocode:

```
best = curr = 0
for each n in nums:
curr = curr + 1 if n == 1 else 0
best = max(best, curr)
return best
```

Time: O(n). Space: O(1)."

Key insight: a running streak (increment on 1, reset to 0 on 0) replaces the nested restart loop — a single pass with no inner scan.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach.

```
class _485_MaxConsecutiveOnes:
    def findMaxConsecutiveOnes(self, nums: List[int]) -> i
nt:
        best = curr = 0
        for n in nums:
            curr = curr + 1 if n == 1 else 0
            best = max(best, curr)
        return best
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

nums=[1,1,0,1,1,1]:

n=1: curr=1,best=1. n=1: curr=2,best=2. n=0: curr=0.

n=1: curr=1. n=1: curr=2. n=1: curr=3,best=3. return 3 ok

nums=[0]: curr=0 throughout. return 0 ok

nums=[1,1,1]: curr counts to 3. best=3 ok

Resets cleanly on zeros and tracks the best streak

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) single pass	O(1) two counters

curr is enough because we don't need to remember WHERE the streak started, only HOW LONG it is.

Follow-up: Max Consecutive Ones II (LC 487) — flip at most one 0; sliding window tracking zero count.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: Max Consecutive Ones III (LC 1004) — flip at most k zeros;

same sliding window, evict the leftmost when zero-count exceeds k.

IN YOUR APP | Expose | Editorial | Community

EASY | Arrays & Hashing | LeetCode #705 | 9 of 19 | 9/19

Design HashSet

QUESTION

Design a HashSet without using any built-in hash table libraries.

Implement MyHashSet class:

CONSTRAINTS

- void add(key) Inserts the value key into the HashSet.
- bool contains(key) Returns whether the value key exists in the HashSet or not.
- void remove(key) Removes the value key in the HashSet. If key does not exist in the HashSet, do nothing.
- 0 <= key <= 10^6
- At most 10^4 calls will be made to add, remove, and contains.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Implement a HashSet without using built-in hash table libraries: add, remove, contains. Right?"

"A few quick questions:

Need to handle arbitrary non-negative ints up to some bound (per constraints)? Yes."

"Let me walk: add(1), add(2), contains(1)->true, remove(2), contains(2)->false ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
keep a boolean array with one slot for every possible
    key (0 to 10^6)
add, remove, contains read or write that slot
```

TIME	SPACE
O(1)	O(10^6)

WHY IT IS SLOW

Constant time but memory for every possible key even if few are stored. Hash into a fixed number of buckets (lists) with modulo: O(n) space and O(1) average operations.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Array of linked lists (chaining) — O(1) amortized. Walk-through:"

`buckets = [[] for _ in range(1000)].`

`hash(key) = key % 1000.`

`add(key): b=hash(key). if key not in buckets[b]: buckets[b].append(key).`

PHASE 3 OPTIMIZE

`remove(key): b=hash(key). if key in buckets[b]: buckets[b].remove(key).`

`contains(key): return key in buckets[b].`

WHY chaining: multiple keys can hash to same bucket -> store as list.

WHY 1000 buckets: balances memory vs collision rate. For n distinct keys, avg O(n/1000) per bucket.

Alternative: open addressing (linear probe) — more complex but better cache performance.

Time: O(1) amortized (O(n) worst per op if all keys collide). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _705_DesignHashSet:
    def __init__(self):
        self.num_buckets = 769
        self.buckets = [[] for _ in range(self.num_buckets
)]

    def _hash(self, key):
        return key % self.num_buckets

    def add(self, key):
        bucket = self.buckets[self._hash(key)]
        if key not in bucket:
            bucket.append(key)

    def remove(self, key):
        bucket = self.buckets[self._hash(key)]
        if key in bucket:
            bucket.remove(key)

    def contains(self, key):
        return key in self.buckets[self._hash(key)]
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

`add(1): hash(1)=1%769=1. buckets[1]=[1]. add(2): buckets[2]=[2] (different bucket, no collision here).`

`contains(1): hash=1, 1 in buckets[1] -> True ok. remove(2): buckets[2].remove(2)->[]. contains(2)->False ok`

769 is a prime number chosen to reduce hash collisions; bucket lists handle collisions via chaining

Edge: add(key) twice -> second add is idempotent (check contains before appending). remove(key) not present -> no-op. contains on empty set -> False ok.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n/buckets) average per op, O(n) worst case if all keys collide	**O(n + buckets)**

Follow-up: Design HashMap (#706) — same bucket structure but stores key-value pairs.

Follow-up: implement dynamic resizing when load factor exceeds a threshold.

IN YOUR APP | Expose | Editorial | Community

EASY | Arrays & Hashing | LeetCode #724 | 10 of 19 | 10/19

Find Pivot Index

QUESTION

Given an array of integers nums, calculate the pivot index of this array.

The pivot index is the index where the sum of all the numbers strictly to the left of the index is equal to the sum of all the numbers strictly to the index's right.

If the index is on the left edge of the array, then the left sum is 0 because there are no elements to the left. This also applies to the right edge of the array.

Return the leftmost pivot index. If no such index exists, return -1.

Example 1

Input	`nums = [1,7,3,6,5,6]`
Output	`3`
Explanation	*The pivot index is 3.* *Left sum = nums[0] + nums[1] + nums[2] = 1 + 7 + 3 = 11* *Right sum = nums[4] + nums[5] = 5 + 6 = 11*

Example 2

Input	`nums = [1,2,3]`
Output	`-1`
Explanation	*There is no index that satisfies the conditions in the problem statement.*

Example 3

Input	`nums = [2,1,-1]`
Output	`0`
Explanation	*The pivot index is 0.* *Left sum = 0 (no elements to the left of index 0)* *Right sum = nums[1] + nums[2] = 1 + -1 = 0*

CONSTRAINTS

- 1 <= nums.length <= 10^4
- 1000 <= nums[i] <= 1000
- Note: This question is the same as 1991: https://leetcode.com/problems/find-the-middle-index-in-array/

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Find the pivot index where sum(left) == sum(right). Return -1 if none. Right?"

"A few quick questions:

PHASE 1 CLARIFY

Elements to the left of index 0 = 0 (empty sum)? Yes. Multiple pivots? Return leftmost."

"Let me walk: nums=[1,7,3,6,5,6] -> index 3: left=[1,7,3]=11, right=[5,6]=11 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    sum the elements to its left and to its right
        with two loops
return the first i where they are equal
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

Re-sums both sides for every index. With the total sum, keep a running left sum and check left == total - left - nums[i]: O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Two-pointer prefix sum check — O(n). Walk-through:"

`total = sum(nums). left_sum=0.`

For i in 0..n-1:

`right_sum = total - left_sum - nums[i].`

If left_sum == right_sum: return i.

`left_sum += nums[i].`

Return -1.

WHY left_sum == right_sum: pivot means sum of elements to left equals sum to right (exclude pivot).

`right_sum = total - left_sum - nums[i]. (total minus left minus pivot itself)`

WHY no extra array: maintain running left_sum and compute right_sum on the fly.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _724_FindPivotIndex:
    def pivotIndex(self, nums):
        total = sum(nums)
        left_sum = 0
        for i, num in enumerate(nums):
            if left_sum == total - left_sum - num:
                return i
            left_sum += num
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

nums=[1,7,3,6,5,6]: total=28.

i=0: left=0, right=28-0-1=27. 0!=27. left=1.

i=1: 1, 28-1-7=20. 1!=20. left=8.

i=2: 8, 28-8-3=17. 8!=17. left=11.

i=3: 11, 28-11-6=11. 11==11 -> return 3 ok

Check BEFORE updating left_sum keeps the index strictly to the left

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) two passes (one for total, one scan)	**O(1)**

Follow-up: if multiple valid pivots, collect them all — remove the 'return i', add to list.

Follow-up: Subarray Sum Equals K (#560) — generalise to any target sum difference.

EASY | Arrays & Hashing | LeetCode #760 | 11 of 19 | 11/19

Find Anagram Mappings

QUESTION

You are given two integer arrays nums1 and nums2 where nums2 is an anagram of nums1. Both arrays may contain duplicates.

Return an index mapping array mapping from nums1 to nums2 where mapping[i] = j means the ith element in nums1 appears in nums2 at index j. If there are multiple answers, return any of them.

An array a is an anagram of an array b means b is made by randomizing the order of the elements in a.

Example 1	
Input	`nums1 = [12,28,46,32,50], nums2 = [50,12,32,46,28]`
Output	`[1,4,3,2,0]`
Explanation	*As mapping[0] = 1 because the 0th element of nums1 appears at nums2[1], and mapping[1] = 4 because the 1st element of nums1 appears at nums2[4], and so on.*

Example 2	
Input	`nums1 = [84,46], nums2 = [84,46]`
Output	`[0,1]`

CONSTRAINTS

- 1 <= nums1.length <= 100
- nums2.length == nums1.length
- 0 <= nums1[i], nums2[i] <= 10^5
- nums2 is an anagram of nums1.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We have two arrays nums1 and nums2. nums2 is an anagram of nums1 —

same elements, different order. We need to return a mapping array

where mapping[i] is the index in nums2 where nums1[i] appears. Right?"

"A few quick questions:

Can there be duplicate values? The problem says yes.

If a value appears multiple times in nums2, which index do we return?

The problem says any valid answer is accepted — good, that simplifies things.

Both arrays are the same length and nums2 is guaranteed to be an anagram,

so every element in nums1 is guaranteed to exist in nums2?"

PHASE 1 CLARIFY

"Let me walk the example:

nums1=[12,28,46,32,50], nums2=[50,12,32,46,28].

12 is at index 1 in nums2 -> mapping[0]=1.

28 is at index 4 -> mapping[1]=4.

46 is at index 3 -> mapping[2]=3.

32 is at index 2 -> mapping[3]=2.

50 is at index 0 -> mapping[4]=0.

Output: [1,4,3,2,0]. Got it.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for element in nums1:
    scan nums2 for its position
    record the index
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

A scan for each element. A hash map from value to index in nums2 makes each lookup O(1): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is scanning nums2 from scratch for every element in nums1.

We're asking the same question repeatedly: where does this value live in nums2?

A hash map lets us answer that in O(1).

I'll preprocess nums2 once: build a map from value -> index.

Then for each element in nums1, look it up directly.

One thing to flag: if there are duplicates, building the map will overwrite

earlier indices with later ones. Since the problem allows any valid answer,

that's fine — we just return whichever index was stored last.

Pseudocode:

```
index_in_nums2 = {val: i for i, val in
  enumerate(nums2)}
return [index_in_nums2[num] for num in nums1]
```

Time: O(n). Space: O(n) for the hash map.

Does that make sense before I code it?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _760_AnagramMappings:
    def anagramMappings(self, nums1: List[int], nums2: List[int]) -> List[int]:
        index_in_nums2 = {val: i for i, val in enumerate(nums2)}
        return [index_in_nums2[num] for num in nums1]
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

"Let me trace through both examples."

nums1=[12,28,46,32,50], nums2=[50,12,32,46,28]

`index_in_nums2 = {50:0, 12:1, 32:2, 46:3, 28:4}`

nums1[0]=12 -> 1

nums1[1]=28 -> 4

nums1[2]=46 -> 3

nums1[3]=32 -> 2

nums1[4]=50 -> 0

Output: [1,4,3,2,0] ok

nums1=[84,46], nums2=[84,46]

`index_in_nums2 = {84:0, 46:1}`

Output: [0,1] ok

Duplicate case — nums1=[3,3], nums2=[3,3]

`enumerate(nums2): (0,3) -> map={3:0}, then (1,3) -> map={3:1} (overwritten).`

nums1[0]=3 -> 1, nums1[1]=3 -> 1. Output: [1,1].

Both point to the same index — valid since any answer is accepted ok

Duplicate overwrite is intentional and within spec

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(n)**
one pass to build the map, one pass to build the result	for the hash map

The duplicate handling is worth calling out explicitly:

by iterating enumerate(nums2) forward, later indices overwrite earlier ones.

If the interviewer wanted ALL valid indices for duplicates returned,

I'd change the map to store a list of indices per value and pop one per lookup.

Trade-off: if n is tiny (say n=5 like in the example), the brute force $O(n^2)$

is barely slower and uses no extra space. The hash map pays off as n grows.

PHASE 6 COMPLEXITY & FOLLOW-UP

Given more time I'd ask: what if we need to return the mapping in sorted order

by nums2 index? We'd sort the output — O(n log n) — but that changes the spec.

IN YOUR APP | Expose | Editorial | Community

EASY | Arrays & Hashing | LeetCode #953 | 12 of 19 | 12/19

Verifying an Alien Dictionary

QUESTION

In an alien language, surprisingly, they also use English lowercase letters, but possibly in a different order. The order of the alphabet is some permutation of lowercase letters.

Given a sequence of words written in the alien language, and the order of the alphabet, return true if and only if the given words are sorted lexicographically in this alien language.

Example 1

Input	`words = ["hello","leetcode"], order = "hlabcdefgijkmnopqrstuvwxyz"`
Output	`true`
Explanation	*As 'h' comes before 'l' in this language, then the sequence is sorted.*

Example 2

Input	`words = ["word","world","row"], order = "worldabcefghijkmnpqstuvxyz"`
Output	`false`
Explanation	*As 'd' comes after 'l' in this language, then words[0] > words[1], hence the sequence is unsorted.*

Example 3

Input	`words = ["apple","app"], order = "abcdefghijklmnopqrstuvwxyz"`
Output	`false`
Explanation	*The first three characters "app" match, and the second string is shorter (in size.) According to lexicographical rules "apple" > "app", because 'l' > '', where '' is defined as the blank character which is less than any other character (More info).*

CONSTRAINTS

- 1 <= words.length <= 100
- 1 <= words[i].length <= 20
- order.length == 26
- All characters in words[i] and order are English lowercase letters.

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given a custom alphabet order, verify if words are sorted according to that order. Right?"

"A few quick questions:

Standard lexicographic comparison rules apply (shorter prefix word comes first)? Yes."

"Let me walk: words=['hello','leetcode'], order='hlabcdefgijkmnopqrstuvwxyz' -> true (h before l) ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for (i < j) in pairs(words):
    compare them letter by letter under the
        custom alphabet
return False if the earlier is greater
```

TIME	SPACE
O(n^2 * L)	O(1)

WHY IT IS SLOW

Compares all pairs. Sortedness is transitive, so comparing only neighbouring words is enough: O(total characters) with a rank map of the alphabet.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

*"Build char-order map, compare adjacent strings — O(n * L). Walk-through:"*

`order_map = {c: i for i,c in enumerate(order)}.`

For each adjacent pair (words[i], words[i+1]):

Compare character by character.

Find first position where they differ: if order_map[a] > order_map[b]: return False.

If one string is a prefix of the other AND words[i] is longer: return False.

(E.g., 'apple' before 'app' is wrong — longer string can't come first if it's a prefix)

Return True.

WHY check only adjacent pairs: if all adjacent pairs are in order, the whole list is sorted.

WHY prefix check: 'abc' > 'ab' in standard lex; with alien order, if shorter is prefix of longer,

the shorter should come first -> longer first = out of order.

Time: O(n * L). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _953_VerifyingAnAlienDictionary:
    def isAlienSorted(self, words, order):
        rank = {ch: i for i, ch in enumerate(order)}
        def in_order(word1, word2):
            for c1, c2 in zip(word1, word2):
                if rank[c1] != rank[c2]:
                    return rank[c1] < rank[c2]
            return len(word1) <= len(word2)
        return all(in_order(words[i], words[i+1]) for i in
 range(len(words) - 1))
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

words=['hello','leetcode'], order='hlabcdefgijkmnopqrstuvwxyz': rank['h']=0, rank['l']=1.

in_order('hello','leetcode'): c1='h',c2='l': rank[h]=0 != rank[l]=1 -> return 0<1=True ok

words=['word','world','row'], order='worldabcefghijkmnpqstuvxyz': in_order('word','world'):

'w'==,'o'==,'r'==,'d' vs 'l': rank[d] vs rank[l] in custom order -> determines result correctly

Comparing by rank instead of raw character lets any custom alphabet ordering work transparently

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(total characters across all words)	**O(1)** extra beyond the rank map

Follow-up: Verifying an Alien Dictionary II — handle case-insensitivity or extra rules.

Follow-up: Alien Dictionary (#269) — INFER the order from a sorted word list (topological sort), the reverse problem.

EASY | Arrays & Hashing | LeetCode #997 | 13 of 19 | 13/19

Find the Town Judge

QUESTION

In a town, there are n people labeled from 1 to n. There is a rumor that one of these people is secretly the town judge.

If the town judge exists, then:

CONSTRAINTS

- 1. The town judge trusts nobody.
- 2. Everybody (except for the town judge) trusts the town judge.
- 3. There is exactly one person that satisfies properties 1 and 2.

You are given an array trust where trust[i] = [a_i, b_i] representing that the person labeled a_i trusts the person labeled b_i. If a trust relationship does not exist in trust array, then such a trust relationship does not exist.

Return the label of the town judge if the town judge exists and can be identified, or return -1 otherwise.

Example 1

Input	`n = 2, trust = [[1,2]]`
Output	`2`

Example 2

Input	`n = 3, trust = [[1,3],[2,3]]`
Output	`3`

Example 3

Input	`n = 3, trust = [[1,3],[2,3],[3,1]]`
Output	`-1`

- 1 <= n <= 1000
- 0 <= trust.length <= 10^4
- trust[i].length == 2
- All the pairs of trust are unique.
- a_i != b_i
- 1 <= a_i, b_i <= n

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

In a town of n people, the judge trusts nobody but is trusted by everyone else. Find the judge's label,

or -1 if no such person exists. Right?"

"A few quick questions:

trust=[a,b] means a trusts b (directed edge)? Yes."

"Let me walk: n=3, trust=[[1,3],[2,3]] -> person 3 trusts nobody, trusted by 1 and 2 -> judge=3 ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for person in people:
    scan the trust list to check they trust
        nobody and everybody else trusts them
```

TIME	SPACE
O(n * T)	O(1)

WHY IT IS SLOW

A scan of all T trust pairs per person. One pass counting out-degree and in-degree: the judge has in-degree n - 1 and out-degree 0, so O(n + T).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"In-degree/out-degree check — O(n). Walk-through:"

`in_degree = [0]*n. out_degree = [0]*n.`

For (a,b) in trust: in_degree[b]++; out_degree[a]++.

For each person i:

If in_degree[i]==n-1 AND out_degree[i]==0: return i+1.

Return -1.

WHY in_degree == n-1: everyone trusts the judge (n-1 others).

WHY out_degree == 0: the judge trusts nobody.

WHY n-1 (not n): the judge doesn't trust themselves.

Alternative: net_trust = in_degree - out_degree. Judge has net n-1 (trusted by all, trusts none).

Time: O(V + E). Space: O(n).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _997_FindTheTownJudge:
    def findJudge(self, n, trust):
        score = [0] * (n + 1)
        for a, b in trust:
            score[a] -= 1
            score[b] += 1
        for person in range(1, n + 1):
            if score[person] == n - 1:
                return person
        return -1
```

PHASE 5 TEST & DEBUG

"Let me trace findJudge(n=3, trust=[[1,3],[2,3]])."

For each person: compute trust_count (how many they trust) and trusted_by_count

Person 1: trusts 3 -> trust_count[1]=1; Person 2: trusts 3 -> trust_count[2]=1

PHASE 5 TEST & DEBUG

trusted_by_count[3]=2 (both 1 and 2 trust 3)

Judge candidate: trust_count==0 AND trusted_by_count==n-1

Person 3: trust_count[3]=0 AND trusted_by_count[3]=2=n-1=2 -> return 3 ok

WHY in/out degree: judge has in-degree n-1 (everyone trusts them) and out-degree 0.

Edge: n=1 with no trust pairs -> person 1 is judge (trivially trusted by all 0 others).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n + E)	**O(n)**

Follow-up: Find Eventual Safe States (#802) — different graph property (no cycles reachable).

Follow-up: if multiple judges were possible, return all of them — same scan, collect instead of early return.

IN YOUR APP | Expose | Editorial | Community

EASY | Arrays & Hashing | LeetCode #1426 | 14 of 19 | 14/19

Counting Elements

QUESTION

Given an integer array arr, count how many elements x there are, such that x + 1 is also in arr. If there are duplicates in arr, count them separately.

Example 1	
Input	`arr = [1,2,3]`
Output	`2`
Explanation	*1 and 2 are counted cause 2 and 3 are in arr.*

Example 2	
Input	`arr = [1,1,3,3,5,5,7,7]`
Output	`0`
Explanation	*No numbers are counted, cause there is no 2, 4, 6, or 8 in arr.*

CONSTRAINTS

- 1 <= arr.length <= 1000
- 0 <= arr[i] <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

For each element x in the array, we check whether x+1 also exists in the array.

If it does, we count x. And if x appears multiple times, we count each occurrence

separately — so duplicates all contribute individually. Right?"

"A few quick questions:

Can the array be empty? Constraints say length >= 1, so no.

Can values be 0? Yes — so x=0 counts if 1 is present.

Can values be negative? Constraints say 0 <= arr[i] <= 1000, so no.

Are there duplicates? Yes, and the problem explicitly says to count them separately."

"Let me walk the examples:

arr=[1,2,3]: 1+1=2 is in arr -> count 1. 2+1=3 is in arr -> count 2. 3+1=4 not in arr.

Output: 2 ok

arr=[1,1,3,3,5,5,7,7]: 1+1=2 not in arr. 3+1=4 not in arr. Same for 5,7.

Output: 0 ok Got it.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for x in arr:
    scan the array for x + 1
    count the x that have one
```

TIME	SPACE
O(n^2)	O(1)

WHY IT IS SLOW

A scan per element. Put the values in a hash set and test x + 1 for each element (duplicates count separately): O(n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

The bottleneck is the inner scan — we're asking 'does x+1 exist?' n times,

each taking O(n). A hash set answers that in O(1).

But there's a subtlety: duplicates. If arr=[1,1,2], both 1s should be counted.

A plain set tells us x+1 exists but not how many times x appears.

So I'll use a Counter instead — it gives me O(1) existence checks

AND stores the frequency of each value.

Walk over the unique keys in the counter. If key+1 is also in the counter,

add counter[key] to the answer — that accounts for all duplicates of key at once.

Pseudocode:

```
freq = Counter(arr)
for x in freq:
if x+1 in freq: ans += freq[x]
return ans
```

Time: O(n) to build the Counter, O(k) to iterate unique keys where k <= n.

Overall O(n). Space: O(k) for the Counter. Does that make sense?

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1426_CountingElements:
    def countElements(self, arr: List[int]) -> int:
        freq = Counter(arr)
        count = 0
        for x in freq:
            if x + 1 in freq:
                count += freq[x]
        return count
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

arr=[1,2,3]

freq={1:1, 2:1, 3:1}

x=1: 2 in freq -> count+=1. count=1.

x=2: 3 in freq -> count+=1. count=2.

x=3: 4 not in freq.

```
return 2 ok
```

arr=[1,1,3,3,5,5,7,7]

freq={1:2, 3:2, 5:2, 7:2}

x=1: 2 not in freq.

x=3: 4 not in freq.

x=5: 6 not in freq.

x=7: 8 not in freq.

```
return 0 ok
```

Duplicate counting — arr=[1,1,2]

freq={1:2, 2:1}

x=1: 2 in freq -> count+=freq[1]=2. count=2.

x=2: 3 not in freq.

```
return 2 ok (both 1s counted, as the problem requires)
```

Duplicate handling confirmed

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	**O(k)**
one pass to build the Counter, one pass over unique keys	where k is the number of distinct values, at most O(n)

Alternative: use a plain set instead of a Counter, then iterate the original arr.

```
present = set(arr)
return sum(1 for x in arr if x+1 in present)
```

Same O(n) time and O(n) space — slightly simpler but both are valid.

The Counter approach avoids re-iterating arr which is a minor readability win.

Trade-off: if the value range is small and fixed (like 0-1000 here),

a boolean array of size 1001 works as a lookup in O(1) with better cache

performance than a hash map, and O(1001) ~ O(1) space in practice.

Given more time I'd ask: what if we needed to return the actual elements

that qualify rather than a count? We'd collect them into a list instead.

EASY | Arrays & Hashing | LeetCode #1470 | 15 of 19 | 15/19

Shuffle the Array

QUESTION

Given the array nums consisting of 2n elements in the form $[x_1,x_2,...,x_n,y_1,y_2,...,y_n]$.

Return the array in the form $[x_1,y_1,x_2,y_2,...,x_n,y_n]$.

Example 1

Input	`nums = [2,5,1,3,4,7], n = 3`
Output	`[2,3,5,4,1,7]`
Explanation	*Since x_1=2, x_2=5, x_3=1, y_1=3, y_2=4, y_3=7 then the answer is [2,3,5,4,1,7].*

Example 2

Input	`nums = [1,2,3,4,4,3,2,1], n = 4`
Output	`[1,4,2,3,3,2,4,1]`

Example 3

Input	`nums = [1,1,2,2], n = 2`
Output	`[1,2,1,2]`

CONSTRAINTS

- 1 <= n <= 500
- nums.length == 2n
- 1 <= nums[i] <= 10^3

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Given array nums of 2n elements: [x1,x2,...,xn,y1,y2,...,yn].

Return it as [x1,y1,x2,y2,...,xn,yn] — interleave the two halves. Right?"

"A few quick questions:

n is always exactly half the array length? Yes. In-place or new array? New array is fine."

"Let me walk: nums=[2,5,1,3,4,7], n=3 -> [2,3,5,4,1,7] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
result = a new array
for i in range(n):
    append x[i] then y[i] (y[i] is nums[n + i])
```

TIME	SPACE
O(n)	O(n)

PHASE 2 BRUTE FORCE

NOTE

Not slow in time. The result is a second array; packing each pair into one integer (x in the low bits, y shifted up) lets the shuffle happen in place with O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Approach: optimal approach

"The bottleneck is the additional list comprehension — the zip approach is already O(n).

We can write it as a single pass building the result with index arithmetic:

result[2*i] = nums[i], result[2*i+1] = nums[i+n].

Time: O(n). Space: O(n) for the result.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1470_ShuffleArray:
    def shuffle(self, nums, n):
        result = [0] * (2 * n)
        for i in range(n):
            result[2 * i] = nums[i]
            result[2 * i + 1] = nums[i + n]
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

nums=[2,5,1,3,4,7], n=3:

i=0: result[0]=2, result[1]=3. i=1: result[2]=5, result[3]=4.

i=2: result[4]=1, result[5]=7. -> [2,3,5,4,1,7] ok

nums=[1,2,3,4], n=2:
result[0]=1,result[1]=3,result[2]=2,result[3]=4. -> [1,3,2,4] ok

Index formula 2*i and 2*i+1 correctly interleaves the two halves

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n) single pass	O(n) for the result array

The zip approach is more Pythonic; the index approach generalises to in-place.

Follow-up: in-place shuffle? Use bit manipulation: store both values in one int,

unpack in a second pass — O(n) time, O(1) extra space.

PHASE 6 COMPLEXITY & FOLLOW-UP

Follow-up: Destination City (#1436) — similar split-and-map pattern.

IN YOUR APP | Expose | Editorial | Community

EASY | Arrays & Hashing | LeetCode #1480 | 16 of 19 | 16/19

Running Sum of 1d Array

QUESTION

Given an array nums. We define a running sum of an array as runningSum[i] = sum(nums[0]…nums[i]).

Return the running sum of nums.

Example 1

Input	`nums = [1,2,3,4]`
Output	`[1,3,6,10]`
Explanation	*Running sum is obtained as follows: [1, 1+2, 1+2+3, 1+2+3+4].*

Example 2

Input	`nums = [1,1,1,1,1]`
Output	`[1,2,3,4,5]`
Explanation	*Running sum is obtained as follows: [1, 1+1, 1+1+1, 1+1+1+1, 1+1+1+1+1].*

Example 3

Input	`nums = [3,1,2,10,1]`
Output	`[3,4,6,16,17]`

CONSTRAINTS

- 1 <= nums.length <= 1000
- 10^6 <= nums[i] <= 10^6

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return the running sum: running[i] = sum(nums[0..i]). Right?"

"A few quick questions:

In-place or new array? New array is fine. Length same as input? Yes."

"Let me walk: nums=[1,2,3,4] -> [1,3,6,10] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    add up nums[0..i] with a loop
```

TIME	SPACE
O(n^2)	O(n)

WHY IT IS SLOW

PHASE 2 BRUTE FORCE

Re-adds the prefix for every position. Each running sum is the previous one plus nums[i], computed in place: O(n) time and O(1) extra space.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Single pass prefix sum — O(n). Walk-through:"

`running = 0. For each num:`

`running += num. result.append(running).`

That's it — the running sum IS the prefix sum array.

WHY it's called 'prefix sum': result[i] = nums[0] + nums[1] + ... + nums[i].

Useful pattern: after building, any subarray sum nums[i..j] = prefix[j] - prefix[i-1].

Time: O(n). Space: O(n) for output.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1480_RunningSum1DArray:
    def runningSum(self, nums):
        result = []
        total = 0
        for num in nums:
            total += num
            result.append(total)
        return result
```

PHASE 5 TEST & DEBUG

"Let me trace runningSum([1,2,3,4])."

Prefix sum in-place: nums[i] += nums[i-1]

i=1: nums[1]=2+1=3 -> [1,3,3,4]

i=2: nums[2]=3+3=6 -> [1,3,6,4]

i=3: nums[3]=4+6=10 -> [1,3,6,10]

Return [1,3,6,10] ok

WHY in-place: O(1) extra space; each element becomes cumulative sum.

Alternative: accumulate function from itertools.

Edge: single element -> return [nums[0]].

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(n)

One-liner: itertools.accumulate(nums) — same semantics.

Follow-up: Range Sum Query (#303) — use these prefix sums for O(1) range queries.

Follow-up: if in-place required: nums[i] += nums[i-1] for i in range(1, n).

IN YOUR APP | Expose | Editorial | Community

EASY | Arrays & Hashing | LeetCode #1534 | 17 of 19 | 17/19

Count Good Triplets

QUESTION

Given an array of integers arr, and three integers a, b and c. You need to find the number of good triplets.

A triplet (arr[i], arr[j], arr[k]) is good if the following conditions are true:

CONSTRAINTS

- 0 <= i < j < k < arr.length
- |arr[i] - arr[j]| <= a
- |arr[j] - arr[k]| <= b
- |arr[i] - arr[k]| <= c

Where |x| denotes the absolute value of x.

Return the number of good triplets.

Example 1

Input	`arr = [3,0,1,1,9,7], a = 7, b = 2, c = 3`
Output	`4`
Explanation	*There are 4 good triplets: [(3,0,1), (3,0,1), (3,1,1), (0,1,1)].*

Example 2

Input	`arr = [1,1,2,2,3], a = 0, b = 0, c = 1`
Output	`0`
Explanation	*No triplet satisfies all conditions.*

- 3 <= arr.length <= 100
- 0 <= arr[i] <= 1000
- 0 <= a, b, c <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

We need to count all index triplets (i, j, k) where i < j < k,

and all three pairwise conditions hold simultaneously:

the first pair must differ by at most a,

the middle-to-last pair by at most b,

and the first-to-last pair by at most c.

We're counting triplets, not returning them. Right?"

"A few quick questions:

Can values in arr be 0? Yes, constraints allow it.

Can a, b, c be 0? Yes — that means exact equality required for that pair.

Array length is at most 100 — good to know for complexity decisions."

PHASE 1 CLARIFY

"Let me walk the example:

arr=[3,0,1,1,9,7], a=7, b=2, c=3.

(3,0,1) at indices (0,1,2): |3-0|=3<=7, |0-1|=1<=2, |3-1|=2<=3 ok

(3,0,1) at indices (0,1,3): |3-0|=3<=7, |0-1|=1<=2, |3-1|=2<=3 ok

(3,1,1) at indices (0,2,3): |3-1|=2<=7, |1-1|=0<=2, |3-1|=2<=3 ok

(0,1,1) at indices (1,2,3): |0-1|=1<=7, |1-1|=0<=2, |0-1|=1<=3 ok

Output: 4. Got it.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for i in range(n):
    for j in range(i_+_1,_n):
        for k in range(j_+_1,_n):
            if |arr[i] - arr[j]| <= a and
                |arr[j] - arr[k]| <= b
                and |arr[i] - arr[k]| <=
                c:
                count += 1
```

TIME	SPACE
O(n^3)	O(1)

WHY IT IS SLOW

Fine for n <= 100, which is the constraint. For larger inputs, fix j, count the valid i by prefix counts over values, and combine with the valid k by counting: O(n * range) or O(n log n).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

Approach: optimal approach

"The algorithm stays O(n³) — there's no known sub-cubic solution for this

specific three-condition problem, and with n=100 we don't need one.

But I can make two practical improvements:

First: early pruning. The three conditions are checked in sequence.

If |arr[i] - arr[j]| > a, there's no point checking any k for this (i,j) pair.

We skip the entire inner k loop, reducing the constant factor significantly.

Second: use itertools.combinations for cleaner, more readable code.

It generates all (i,j,k) triples with i<j<k in one line.

Note that combinations doesn't support early pruning on the outer pair,

so for readability I'll use it here since n is small enough that it doesn't matter.

PHASE 3 OPTIMIZE

I'll go with the early-pruning nested loop to show I understand the trade-off,

then mention the itertools alternative.

Time: O(n³) worst case, faster in practice due to pruning.

Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement with early pruning and meaningful names."

itertools one-liner alternative (interview-clean, no pruning):

```
class _1534_CountGoodTriplets:
    def countGoodTriplets(self, arr: List[int], a: int, b:
 int, c: int) -> int:
        count = 0
        n = len(arr)
        for i in range(n):
            for j in range(i + 1, n):
                if abs(arr[i] - arr[j]) > a:   # prune: sk
ip all k for this pair
                    continue
                for k in range(j + 1, n):
                    if (abs(arr[j] - arr[k]) <= b and
                        abs(arr[i] - arr[k]) <= c):
                        count += 1
        return count

class _1534_CountGoodTriplets_Itertools:
    def countGoodTriplets(self, arr: List[int], a: int, b:
 int, c: int) -> int:
        return sum(
            abs(arr[i] - arr[j]) <= a and
            abs(arr[j] - arr[k]) <= b and
            abs(arr[i] - arr[k]) <= c
            for i, j, k in itertools.combinations(range(le
n(arr)), 3)
        )
```

PHASE 5 TEST & DEBUG

"Let me trace the key invariant and walk through the example to show how the hash structure updates."

"Let me trace the pruning version on example 1."

arr=[3,0,1,1,9,7], a=7, b=2, c=3

i=0 (arr[i]=3):

j=1 (arr[j]=0): |3-0|=3<=7 -> not pruned.

k=2: |0-1|=1<=2, |3-1|=2<=3 -> count=1

k=3: |0-1|=1<=2, |3-1|=2<=3 -> count=2

k=4: |0-9|=9>2 -> skip

k=5: |0-7|=7>2 -> skip

j=2 (arr[j]=1): |3-1|=2<=7 -> not pruned.

k=3: |1-1|=0<=2, |3-1|=2<=3 -> count=3

k=4: |1-9|=8>2 -> skip

k=5: |1-7|=6>2 -> skip

j=3 (arr[j]=1): |3-1|=2<=7 -> not pruned.

k=4: |1-9|=8>2 -> skip

PHASE 5 TEST & DEBUG

k=5: |1-7|=6>2 -> skip

j=4 (arr[j]=9): |3-9|=6<=7 -> not pruned.

k=5: |9-7|=2<=2, |3-7|=4>3 -> skip

j=5: no k available.

i=1 (arr[i]=0):

j=2 (arr[j]=1): |0-1|=1<=7 -> not pruned.

k=3: |1-1|=0<=2, |0-1|=1<=3 -> count=4

k=4,5: fail b condition -> skip

... remaining pairs yield 0 more.

`return 4 ok`

arr=[1,1,2,2,3], a=0, b=0, c=1

a=0 means |arr[i]-arr[j]| must be 0 -> only equal pairs pass.

j=1 (1==1): pass a. k=2: |1-2|=1>0 -> fail b. k=3: same. k=4: same.

j=3 (1==?): arr[1]=1, arr[3]=2 -> |1-2|=1>0 -> pruned.

No triplet survives. return 0 ok

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n³) worst case when all pairs pass the first condition —	**O(1)** just a counter

but early pruning on condition 1 cuts the k loop entirely for bad (i,j) pairs,

making it much faster in practice.

With n=100 the worst case is 161,700 triplets — trivially fast.

If n grew to 10^4, O(n³) would be 10^12 — completely infeasible.

At that scale I'd look at sorting + binary search to narrow candidates per (i,j),

or segment trees for range queries — but that's overkill here.

The itertools one-liner is the cleanest version for an interview

when you want to show Python fluency without sacrificing readability.

Trade-off: it can't prune early, so stick with the nested loop if the

interviewer asks about optimization.

Given more time I'd ask: what if the array is sorted?

Sorted order lets us binary search for valid k values after fixing i and j,

reducing the inner loop from O(n) to O(log n) -> overall O(n² log n).

EASY | Arrays & Hashing | LeetCode #1854 | 18 of 19 | 18/19

Maximum Population Year

QUESTION

You are given a 2D integer array logs where each logs[i] = [birth$_i$, death$_i$] indicates the birth and death years of the ith person.

The population of some year x is the number of people alive during that year. The ith person is counted in year x's population if x is in the inclusive range [birth$_i$, death$_i$ - 1]. Note that the person is not counted in the year that they die.

Return the earliest year with the maximum population.

Example 1

Input	`logs = [[1993,1999],[2000,2010]]`
Output	`1993`
Explanation	*The maximum population is 1, and 1993 is the earliest year with this population.*

Example 2

Input	`logs = [[1950,1961],[1960,1971],[1970,1981]]`
Output	`1960`
Explanation	*The maximum population is 2, and it had happened in years 1960 and 1970. The earlier year between them is 1960.*

CONSTRAINTS

- 1 <= logs.length <= 100
- 1950 <= birth$_i$ < death$_i$ <= 2050

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Each person is born in birth year and dies in death year (exclusive — don't count death year).

Find the year with the maximum population. 1950-2050 range. Right?"

"A few quick questions:

Tie: return the earliest year? Yes."

"Let me walk: logs=[[1993,1999],[2000,2010]] -> year 1993-1999: [1993..1998]=1 each. 2000: 1. Max at 1993.

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
for year in range(1950,_2050_+_1):
    count the people alive that year
return the earliest year with the most
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n * years)	O(1)

WHY IT IS SLOW

Scans every person for every year. A difference array over the years (+1 at birth, -1 at death) with a running sum is O(n + years).

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Difference array: +1 at birth year, -1 at death+1, find prefix max — O(n). Walk-through:"

`diff = [0] * (2050 - 1950 + 2). (offset: year - 1950)`

For (birth, death) in logs:

diff[birth-1950] += 1.

diff[death-1950+1] -= 1. <- person is alive through death year, so +1 ends at death+1

Prefix sum: running = 0.

For year in 0..100: running += diff[year]. If running > max_pop: max_pop=running; best_year=year.

Return 1950 + best_year.

WHY difference array: range update [birth, death] in O(1) per person.

WHY prefix sum gives population: each year's population = cumulative sum up to that year.

Time: O(n). Space: O(1).

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```python
class _1854_MaxPopulationYear:
    def maximumPopulation(self, logs):
        BASE = 1950
        delta = [0] * 101
        for birth, death in logs:
            delta[birth - BASE] += 1
            delta[death - BASE] -= 1
        best_pop = best_year = 0
        current = 0
        for i in range(101):
            current += delta[i]
            if current > best_pop:
                best_pop = current
                best_year = i + BASE
        return best_year
```

PHASE 5 TEST & DEBUG

"Let me trace maximumPopulation([[1950,1961],[1960,1971],[1970,1981]])."

Difference array on years 1950-2050

Person 1: diff[1950]+=1, diff[1961]-=1

Person 2: diff[1960]+=1, diff[1971]-=1

PHASE 5 TEST & DEBUG

Person 3: diff[1970]+=1, diff[1981]-=1

Prefix sum: 1950:1, 1951-1959:1, 1960:2, 1961-1969:1, 1970:2, 1971-1980:1

Max population=2 at years 1960 and 1970. Return earliest=1960 ok

WHY difference array: efficient range update; prefix sum gives population per year.

Edge: overlapping ranges -> prefix sum naturally accumulates counts.

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n + 101)	**O(101)**

Difference array is the key pattern for range increment problems.

Follow-up: Car Pooling (#1094) — same difference array for capacity.

Follow-up: if years can be any range, use a hash map instead of array.

IN YOUR APP | Expose | Community

EASY | Arrays & Hashing | LeetCode #1929 | 19 of 19 | 19/19

Concatenation of Array

QUESTION

Given an integer array nums of length n, you want to create an array ans of length 2n where ans[i] == nums[i] and ans[i + n] == nums[i] for 0 <= i < n (0-indexed).

Specifically, ans is the concatenation of two nums arrays.

Return the array ans.

Example 1

Input	`nums = [1,2,1]`
Output	`[1,2,1,1,2,1]`
Explanation	*The array ans is formed as follows: - ans = [nums[0],nums[1],nums[2],nums[0],nums[1],nums[2]] - ans = [1,2,1,1,2,1]*

Example 2

Input	`nums = [1,3,2,1]`
Output	`[1,3,2,1,1,3,2,1]`
Explanation	*The array ans is formed as follows: - ans = [nums[0],nums[1],nums[2],nums[3],nums[0],nums[1],nums[2],nums[3]] - ans = [1,3,2,1,1,3,2,1]*

CONSTRAINTS

- n == nums.length
- 1 <= n <= 1000
- 1 <= nums[i] <= 1000

INTERVIEW APPROACH · STAR-LC

PHASE 1 CLARIFY

Let me make sure I understand the problem correctly.

Return an array ans of length 2n where ans[i]=nums[i] and ans[i+n]=nums[i] for 0<=i<n

(essentially nums concatenated with itself). Right?"

"A few quick questions:

Just a straightforward concatenation, no transformation? Correct."

"Let me walk: nums=[1,2,1] -> [1,2,1,1,2,1] ok

PHASE 2 BRUTE FORCE

"Let me start with brute force to lock in the logic."

BRUTE FORCE · PSEUDO-CODE

```
ans = a new array of length 2n
for i in range(n):
    ans[i] = nums[i] and ans[i + n] = nums[i]
```

PHASE 2 BRUTE FORCE

TIME	SPACE
O(n)	O(n)

NOTE

Not slow: every element is written twice, which is optimal. The output has to be a new array of size 2n.

"Should I go ahead and optimize?"

PHASE 3 OPTIMIZE

"Return nums + nums — O(n). Walk-through:"

`ans = nums + nums (Python list concatenation creates a new list).`

Or: ans = nums * 2.

The concatenation of array is itself twice: [1,2,3] -> [1,2,3,1,2,3].

No complex logic needed — problem is a warm-up/starter.

Time: O(n). Space: O(n) for output."

Key insight: the output is structurally identical to nums+nums — the concatenation IS the solution, no algorithmic transformation required.

PHASE 4 CLEAN CODE

Now I'll implement the optimized approach with meaningful names.

```
class _1929_ConcatenationOfArray:
    def getConcatenation(self, nums):
        return nums + nums
```

PHASE 5 TEST & DEBUG

"Let me trace getConcatenation([1,2,1])."

Simply return nums + nums = [1,2,1,1,2,1] ok

Python: return nums * 2 or nums + nums

Trace [1,3,2,1]: return [1,3,2,1,1,3,2,1] ok

WHY trivial: ans[i]=nums[i] and ans[i+n]=nums[i] for all i in [0,n).

Edge: empty array -> return [] (technically [] + [] = []). Single element: [x,x].

In-place: for i in range(n): ans[i+n]=ans[i]. Or just return nums+nums. O(n) time, O(n) space.

Edge: ans has exactly 2n elements; first n from original, last n identical copy. Trivial O(n).

PHASE 6 COMPLEXITY & FOLLOW-UP

TIME	SPACE
O(n)	O(n)

Follow-up: Build Array from Permutation (#1920) — similar simple array transformation warm-up.

Follow-up: if asked for O(1) extra space (in-place using a doubled array param), just write into a pre-sized result.

IN YOUR APP | Expose | Community